

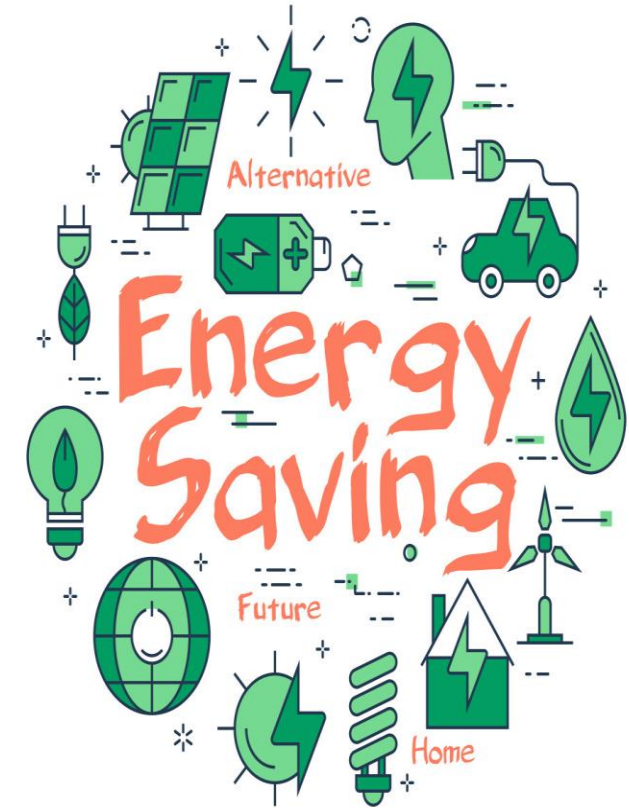
Smart Grid Optimization using Machine Learning

Team-E

Kalakonda Lokesh Setty
Kesari Nandan Mallepally
Ajay Kumar Reddy Kethireddy

Class: DS606 - Data Science Capstone

Professor: Unal Sakoglu



GitHub Repo Overview

- Link: <https://github.com/settylokes/Smart-Grid-Optimization/blob/main/EDA.ipynb>
- Structure:
 - EDA.ipynb
 - Model_Comparison.ipynb
 - Prophet_Forecasting.ipynb
 - README with summary & visuals

The screenshot shows the GitHub repository page for 'Smart-Grid-Optimization' by user 'settylokes'. The repository is public and has 2 branches and 0 tags. The main branch is selected. The file list shows the following files and their commit history:

File	Commit Message	Time
data	added merged_data Updated	18 hours ago
.gitignore	add .gitignore to exclude virtual environment files	last week
EDA.ipynb	updated eda	5 hours ago
README.md	Create README.md	2 months ago
SetUp and Modelling.ipynb	updated	1 hour ago

The README section is visible, showing the title '606-Project'.

On the right side, the 'About' section states: 'No description, website, or topics provided.' Below this, it shows 'Readme', 'Activity', '0 stars', '1 watching', and '0 forks'. The 'Releases' section states: 'No releases published. Create a new release'. The 'Packages' section states: 'No packages published. Publish your first package'.

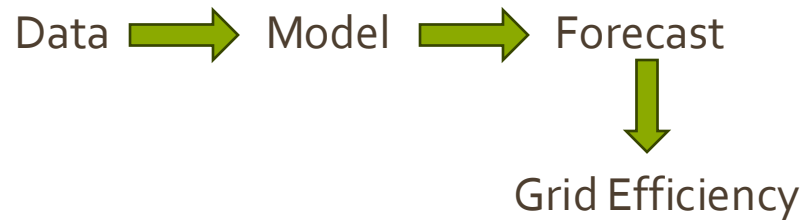
Project Summary

- Objective

- To accurately forecast energy load in the California smart grid system by leveraging historical production and weather data. This supports more efficient grid management, helping reduce renewable energy curtailment and improve resource allocation.

- Task Type

- Supervised Regression problem
- Target Variable: Energy Load
- Input Features: Time, temperature, humidity, solar energy, wind speed, etc.



- Goal

- Train and evaluate multiple machine learning models to predict hourly energy demand.
- Identify the best-performing model based on R^2 score and MAE to ensure high prediction accuracy and generalization.
- Reduce overfitting and improve interpretability where needed.
- Application & Real-World Impact
 - Grid Optimization: Predict demand peaks and troughs to prevent energy wastage.
 - Curtailment Reduction: Match renewable energy production with forecasted load, reducing unnecessary energy dump or curtailment.
 - Policy and Planning: Inform decisions for energy storage, load shifting, and infrastructure development.
 - Sustainability: Improve renewable energy utilization, contributing to a cleaner grid.

Literature Survey Refresher

- Prior studies used ARIMA, SVMs, and deep learning (LSTM)
- [MDPI 2022] used ensemble ML for wind prediction
- [ScienceDirect] emphasized storage for renewables integration
- [ResearchGate 2023] explored hybrid models with DL
- Our Approach:
 - Focused on ensemble ML models (Bagging, Stacking, XGBoost)
 - Tailored for CAISO data characteristic

➤ Existing approaches & Proposed Methods

1. Solar Power Production Forecasting

Dataset Used: California Renewable Production 2010-2018

Feature Selection: Historical solar power production, weather variables (temperature, irradiance)

Modeling Techniques: Time series analysis, machine learning models

Reference:

<https://www.kaggle.com/code/cheedcheed/predictions-on-california-solar-power-production>

2. Wind Power Generation Forecasting

Dataset Used: California Renewable Production 2010-2018

Feature Selection: Wind speed, wind direction, historical wind power production

Modeling Techniques: Regression models, time series forecasting

Reference:

<https://www.kaggle.com/code/cheedcheed/california-wind-power-generation-forecasting>

Dataset Overview

- Our project utilizes integrated datasets that combine detailed energy production and curtailment records with comprehensive weather data, enabling an in-depth analysis of renewable energy integration challenges.

1. Production and Curtailment Data:

- Source:** California Independent System Operator (CAISO)."

- Key Attributes:**

- *Demand Metrics:* Includes actual load and net load (actual load minus wind and solar production).
- *Supply Metrics:* Breakdown by energy source types including solar, wind, biomass, geothermal, small hydropower (renewables), large hydro, nuclear, and thermal.
- *Curtailment Records:* Specific intervals showing solar and wind energy curtailment when production exceeded demand or due to other grid management needs.

- Period:** 2014–2025, providing a rich historical context for analysis."

- Importance:** Allows for precise correlation between energy production volumes and curtailment events, facilitating a deeper understanding of when and why curtailment occurs.

- Dataset Size:** 112MB

- Dataset:** <https://www.caiso.com/about/our-business/managing-the-evolving-grid>

Dataset Overview

2. Weather Dataset:

Source: Visual Crossing Weather."

Dataset Size: 8MB"

Dataset: <https://www.visualcrossing.com/weather-query-builder/>

Entries: 41,639 rows.

Coverage: Hourly data points across several years.

Columns: 24 columns including:

- name: Location name (e.g., California).
- datetime: Date and time of the weather data point.
- temp: Temperature in Celsius.
- feelslike: What the temperature feels like in Celsius.
- dew: Dew point in Celsius.
- humidity: Humidity percentage.
- precip: Precipitation amount in millimeters.
- precipprob: Probability of precipitation.
- preciptype: Type of precipitation (if any).
- snow: Snowfall amount in millimeters.
- snowdepth: Snow depth in millimeters.
- icon: Icon representing weather conditions.
- windgust: Wind gust speed in km/h.
- windspeed: Wind speed in km/h.
- winddir: Wind direction in degrees.
- sealevelpressure: Sea level pressure in millibars.
- cloudcover: Cloud cover percentage.
- visibility: Visibility in kilometers.
- solarradiation: Solar radiation in kW/m².
- solarenergy: Solar energy in kWh/m².
- uvindex: UV index.
- severerisk: Severe risk percentage (only available for some entries).
- conditions: General weather conditions (e.g., Clear, Rain).
- stations: Weather stations reporting the data.

Exploratory Data Analysis(EDA)

production_data_2020.head()

	date	hour	interval	load	solar	wind	net load	renewables	nuclear	large hydro	imports	generation	thermal
0	2020-01-01 00:00:00	1	1	22121.306042	0.0	290.128537	21831.177504	1684.108439	2272.914003	1454.382066	8523.626667	13598.950729	8187.111111
1	2020-01-01 00:05:00	1	2	22119.647656	0.0	285.203357	21834.444300	1658.931626	2273.301338	1357.157727	8385.320000	13726.372839	8436.111111
2	2020-01-01 00:10:00	1	3	22056.574375	0.0	275.933807	21780.640568	1666.341899	2272.750655	1319.864573	8333.773333	13716.481484	8457.111111
3	2020-01-01 00:15:00	1	4	21981.423021	0.0	275.207947	21706.215074	1664.394765	2273.263670	1331.500633	8333.493333	13638.609375	8369.111111
4	2020-01-01 00:20:00	1	5	21901.401719	0.0	277.932034	21623.469684	1670.286625	2273.183646	1302.441129	8255.800000	13636.090573	8390.111111

```
-----
0    date                                105408 non-null  datetime64[ns]
1    hour                                105408 non-null  int64
2    interval                            105408 non-null  int64
3    load                                105408 non-null  float64
4    solar                               105408 non-null  float64
5    wind                                105408 non-null  float64
6    net load                            105408 non-null  float64
7    renewables                          105408 non-null  float64
8    nuclear                             105408 non-null  float64
9    large hydro                         105408 non-null  float64
10   imports                             105407 non-null  float64
11   generation                          105408 non-null  float64
12   thermal                             105408 non-null  float64
13   load less (generation+imports)      105407 non-null  float64
dtypes: datetime64[ns](1), float64(11), int64(2)
memory usage: 11.3 MB
None
```

Exploratory Data Analysis(EDA)

```
weather_data.head()
```

	name	datetime	temp	feelslike	dew	humidity	precip	precipprob	preciptype	snow	snowdepth	windgust	windspee
0	califorina	2020-01-01 00:00:00	32.0	32.0	29.7	91.14	0.0	0	NaN	0.0	1.24	7.4	0.
1	califorina	2020-01-01 01:00:00	31.8	27.8	29.9	92.58	0.0	0	NaN	0.0	1.22	8.3	4.
2	califorina	2020-01-01 02:00:00	30.8	26.7	29.3	94.03	0.0	0	NaN	0.0	1.21	8.7	4.
3	califorina	2020-01-01 03:00:00	31.1	24.3	30.0	95.84	0.0	0	NaN	0.0	1.19	7.4	7.
4	califorina	2020-01-01 04:00:00	31.7	31.7	30.2	93.95	0.0	0	NaN	0.0	1.18	8.3	2.

 weather_data.info()



```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 41639 entries, 0 to 41638
Data columns (total 24 columns):
#   Column              Non-Null Count  Dtype
---  -
0   name                 41639 non-null  object
1   datetime             41639 non-null  object
2   temp                41639 non-null  float64
3   feelslike            41639 non-null  float64
4   dew                  41639 non-null  float64
5   humidity             41639 non-null  float64
6   precip               41639 non-null  float64
7   precipprob           41639 non-null  int64
8   preciptype           2884 non-null   object
9   snow                 41639 non-null  int64
10  snowdepth            41639 non-null  float64
11  windgust              41638 non-null  float64
12  windspeed             41639 non-null  float64
13  winddir               41639 non-null  float64
14  sealevelpressure      41639 non-null  float64
15  cloudcover            41639 non-null  float64
16  visibility            41639 non-null  float64
17  solarradiation        41639 non-null  float64
18  solarenergy           41639 non-null  float64
19  uvindex               41639 non-null  int64
20  severerisk            23875 non-null  float64
21  conditions            41639 non-null  object
22  icon                  41639 non-null  object
23  stations              41639 non-null  object
dtypes: float64(15), int64(3), object(6)
memory usage: 7.6+ MB
```


Null Values In Production data

```
# Find the null values in production_data_2020
production_data_2020.isnull().sum()
```

	0
date	0
hour	0
interval	0
load	0
solar	0
wind	0
net load	0
renewables	0
nuclear	0
large hydro	0
imports	1
generation	0
thermal	0
load less (generation+imports)	1

```
# Find the null values in production_data_2020
production_data_2020.isnull().sum()
```

	0
date	0
hour	0
interval	0
load	0
solar	0
wind	0
net load	0
renewables	0
nuclear	0
large hydro	0
imports	0
generation	0
thermal	0
load less (generation+imports)	0

Merging and Sampling Production data

```
Shape of Production Data 2020 (105408, 14)
Shape of Production Data 2021 (105120, 14)
Shape of Production Data 2022 (105120, 14)
Shape of Production Data 2023 (105120, 14)
Shape of Production Data 2024 (105408, 14)
Shape of Production Data 2025 (8928, 14)
```

Since 2020 and 2024 are leap year there are more records

	date	hour	load	solar	wind	net load	renewables	nuclear	large hydro	imports	generation	
0	2020-01-01 00:00:00	0	261642.173073	0.0	3406.967436	258235.205637	20091.520292	27279.731877	15895.241613	99143.546667	162410.879518	99143.546667
1	2020-01-01 01:00:00	1	252464.832604	0.0	5967.002946	246497.829658	22649.576156	27273.699994	14712.373468	91953.413333	160451.063841	95811.063841
2	2020-01-01 02:00:00	2	245416.002344	0.0	6199.244249	239216.758095	22898.479955	27280.022811	14073.236606	93985.426667	151384.511224	87133.426667
3	2020-01-01 03:00:00	3	241363.031667	0.0	5080.579630	236282.452037	21691.987069	27280.588677	14248.185413	94369.893333	146970.820052	83750.893333
4	2020-01-01 04:00:00	4	239642.357318	0.0	9443.764928	230198.592389	26065.495828	27262.534627	15167.772388	93689.306667	145981.897747	77480.306667

```
Shape of Production Data (44592, 13)
```

Null Vales In Weather Data

```
# Find the null values in weather_data
weather_data.isnull().sum()
```

	0
date	0
temp	0
dew	0
humidity	0
precip	0
snowdepth	0
windgust	158
windspeed	0
winddir	0
sealevelpressure	0
cloudcover	0
visibility	0
solarradiation	0
solarenergy	0
uvindex	0

```
weather_data.isnull().sum()
```

	0
date	0
temp	0
dew	0
humidity	0
precip	0
snowdepth	0
windgust	0
windspeed	0
winddir	0
sealevelpressure	0
cloudcover	0
visibility	0
solarradiation	0
solarenergy	0
uvindex	0

Merging Production and Weather Data

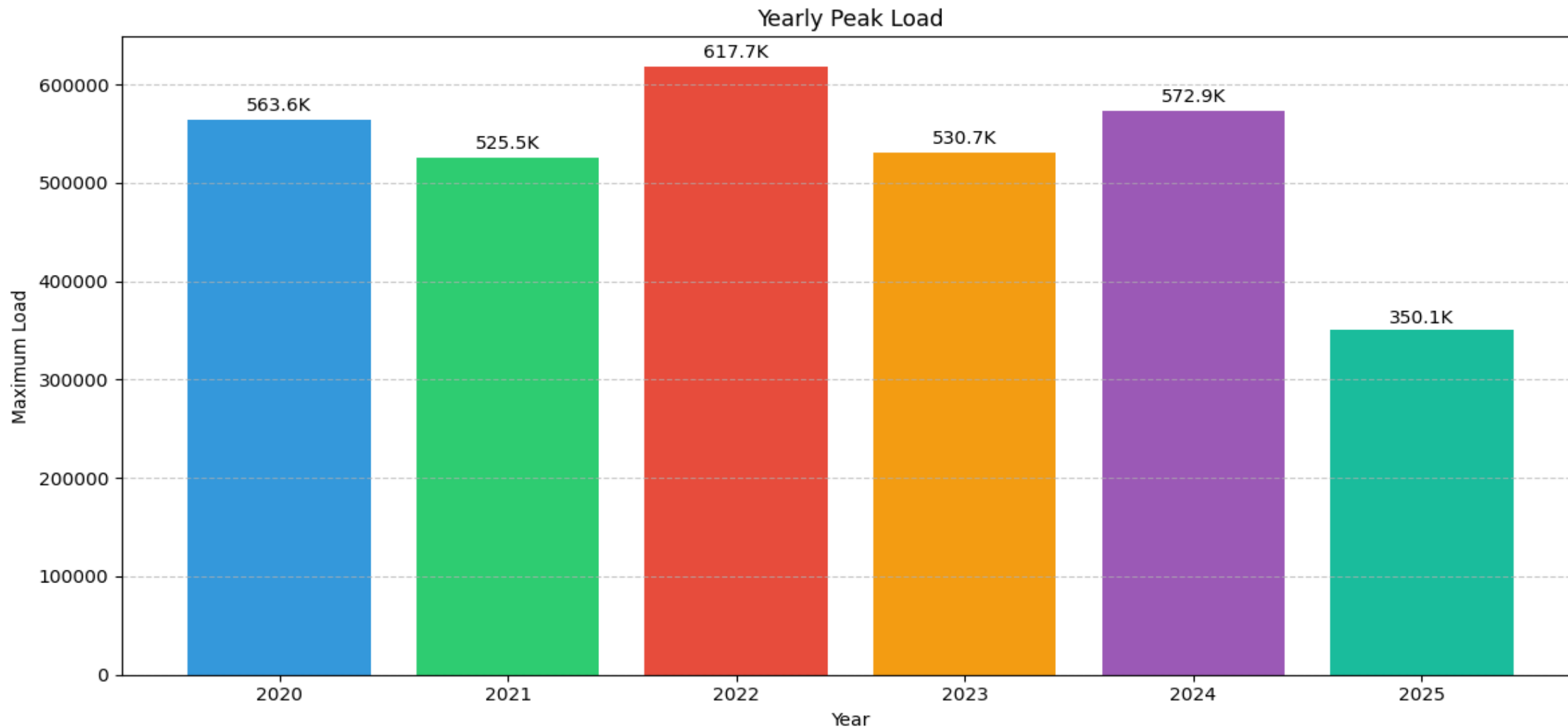
```
# Display the merged DataFrame
merged_data.head()
```

	date	hour	load	solar	wind	net load	renewables	nuclear	large hydro	imports	generation	the
0	2020-01-01 00:00:00	0	261642.173073	0.0	3406.967436	258235.205637	20091.520292	27279.731877	15895.241613	99143.546667	162410.879518	99144.38
1	2020-01-01 01:00:00	1	252464.832604	0.0	5967.002946	246497.829658	22649.576156	27273.699994	14712.373468	91953.413333	160451.063841	95815.41
2	2020-01-01 02:00:00	2	245416.002344	0.0	6199.244249	239216.758095	22898.479955	27280.022811	14073.236606	93985.426667	151384.511224	87132.77
3	2020-01-01 03:00:00	3	241363.031667	0.0	5080.579630	236282.452037	21691.987069	27280.588677	14248.185413	94369.893333	146970.820052	83750.05
4	2020-01-01 04:00:00	4	239642.357318	0.0	9443.764928	230198.592389	26065.495828	27262.534627	15167.772388	93689.306667	145981.897747	77486.09

```
print("Shape of Merged Data",merged_data.shape)
```

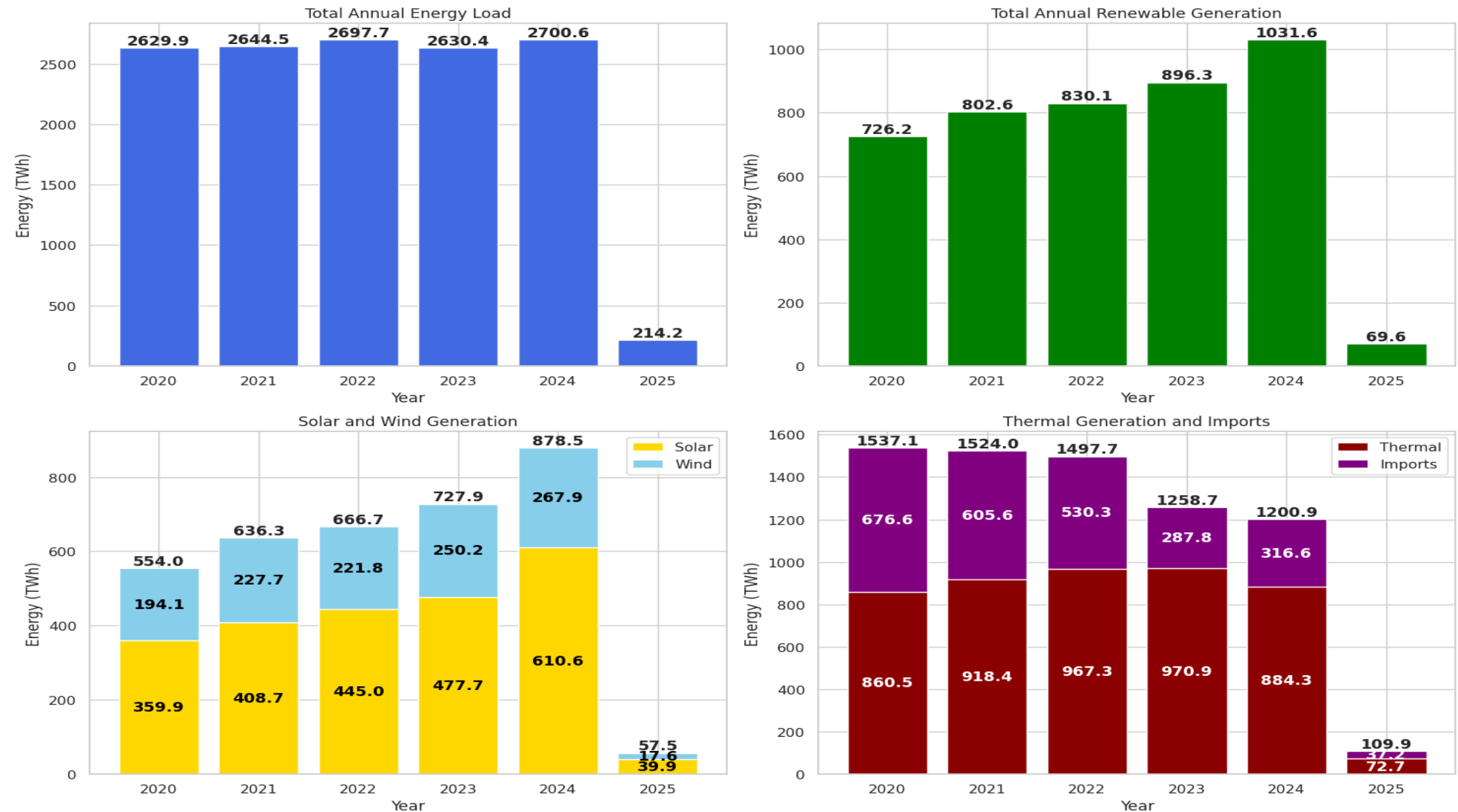
Shape of Merged Data (44592, 27)

Yearly Peak Load

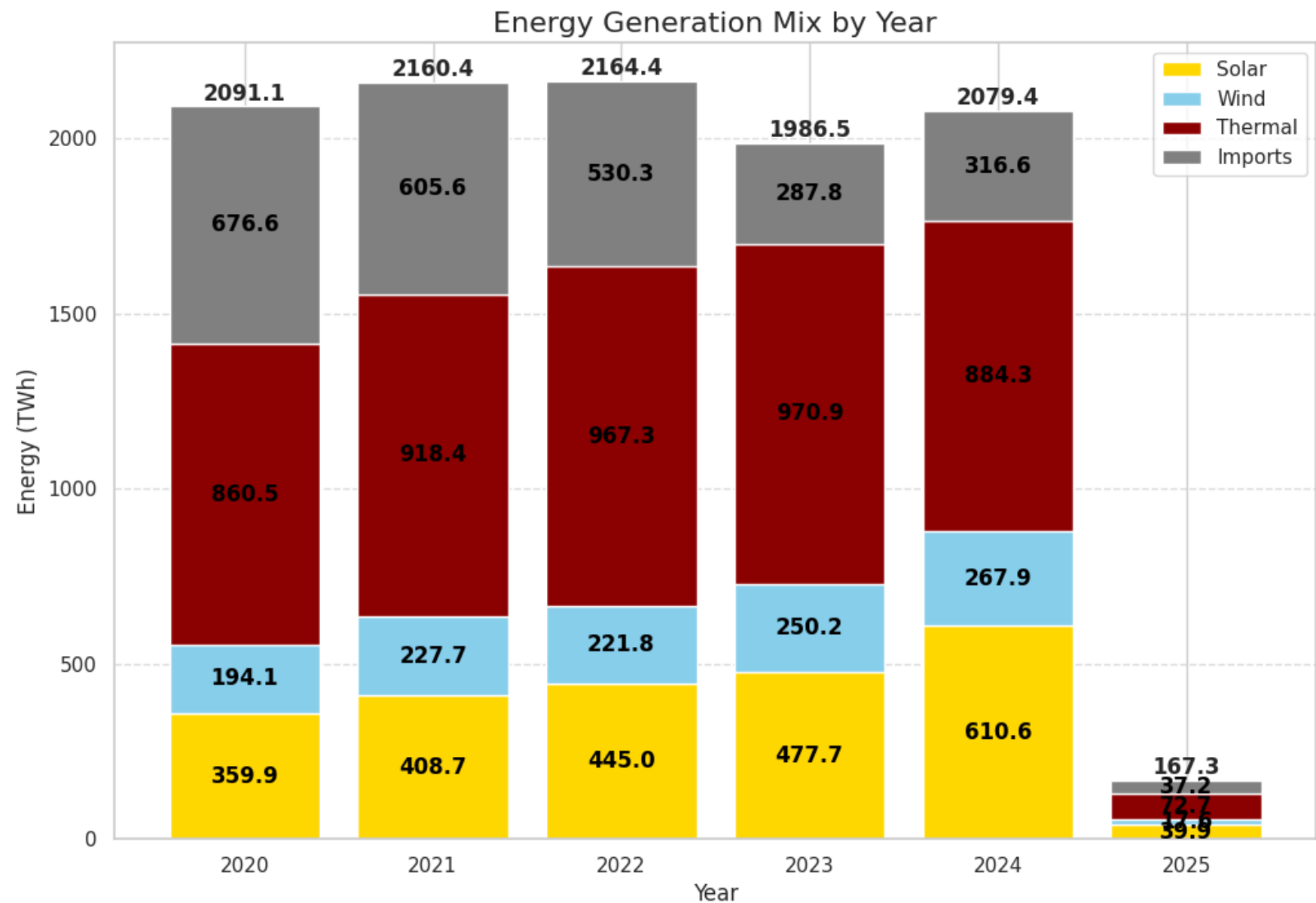


Energy Metrics by Year (2020-2025)

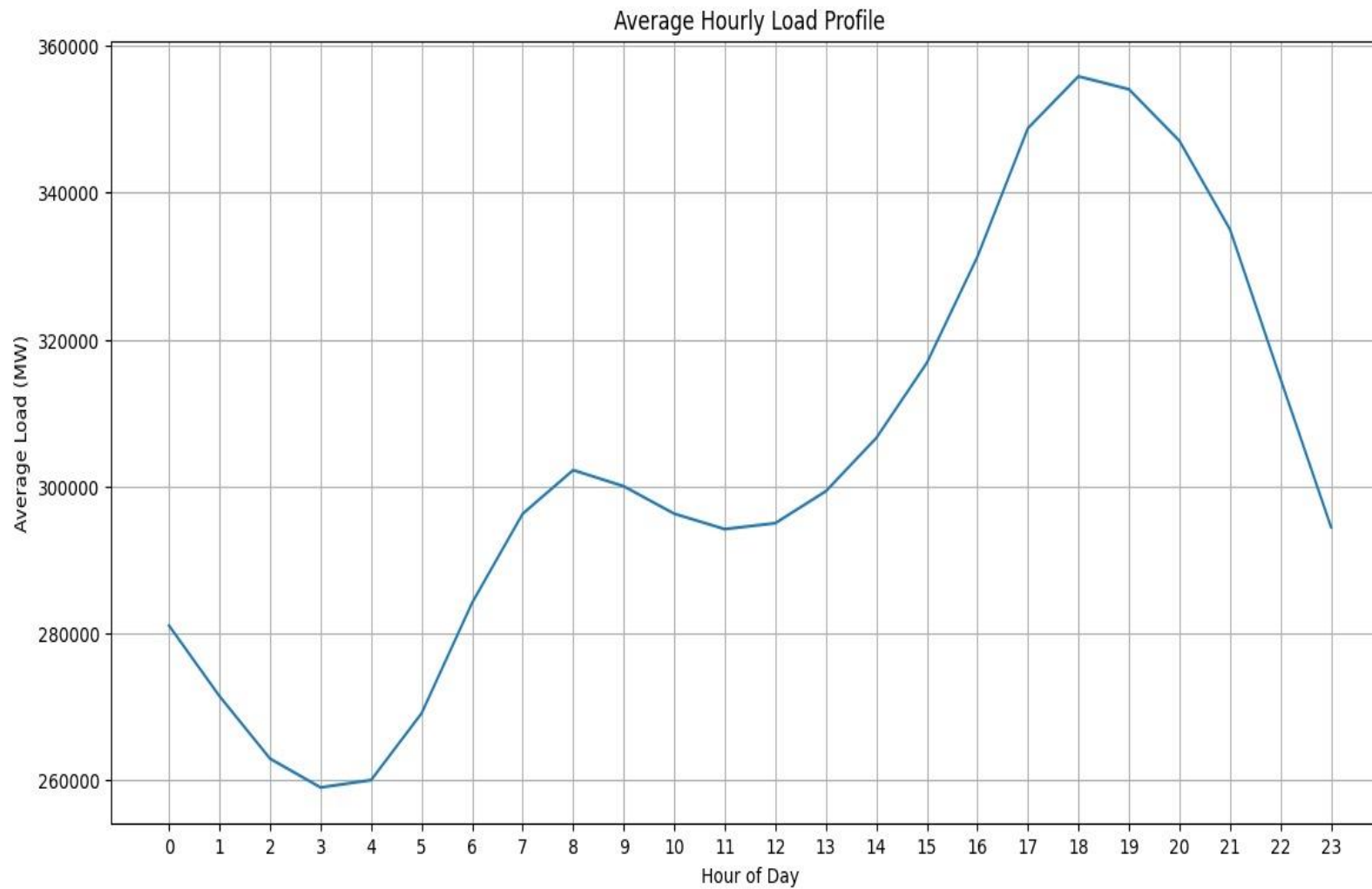
Energy Metrics by Year (2020-2025)



Energy Generation Mix by year

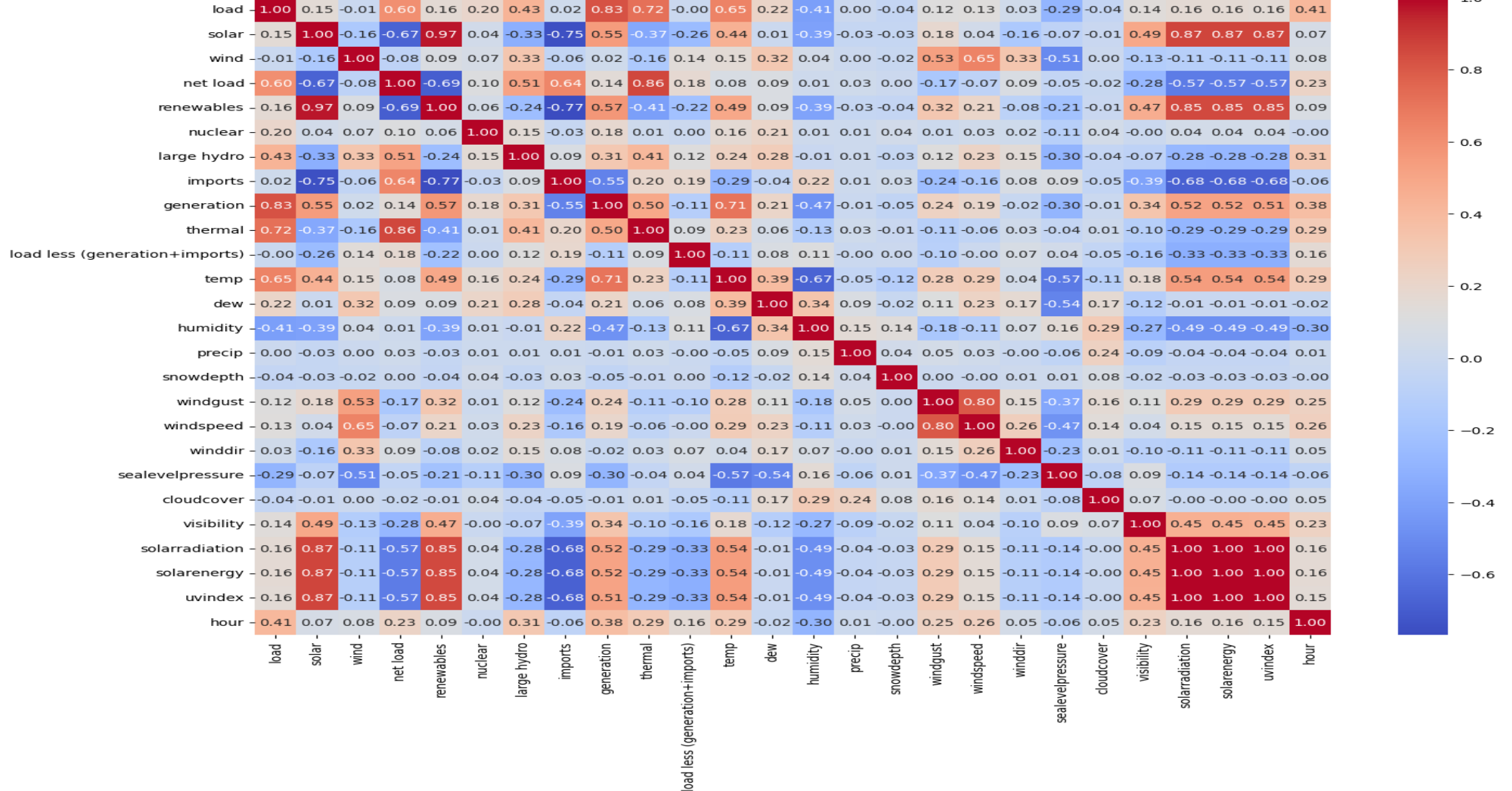


Average Hourly Load Profile



Correlation Matrix

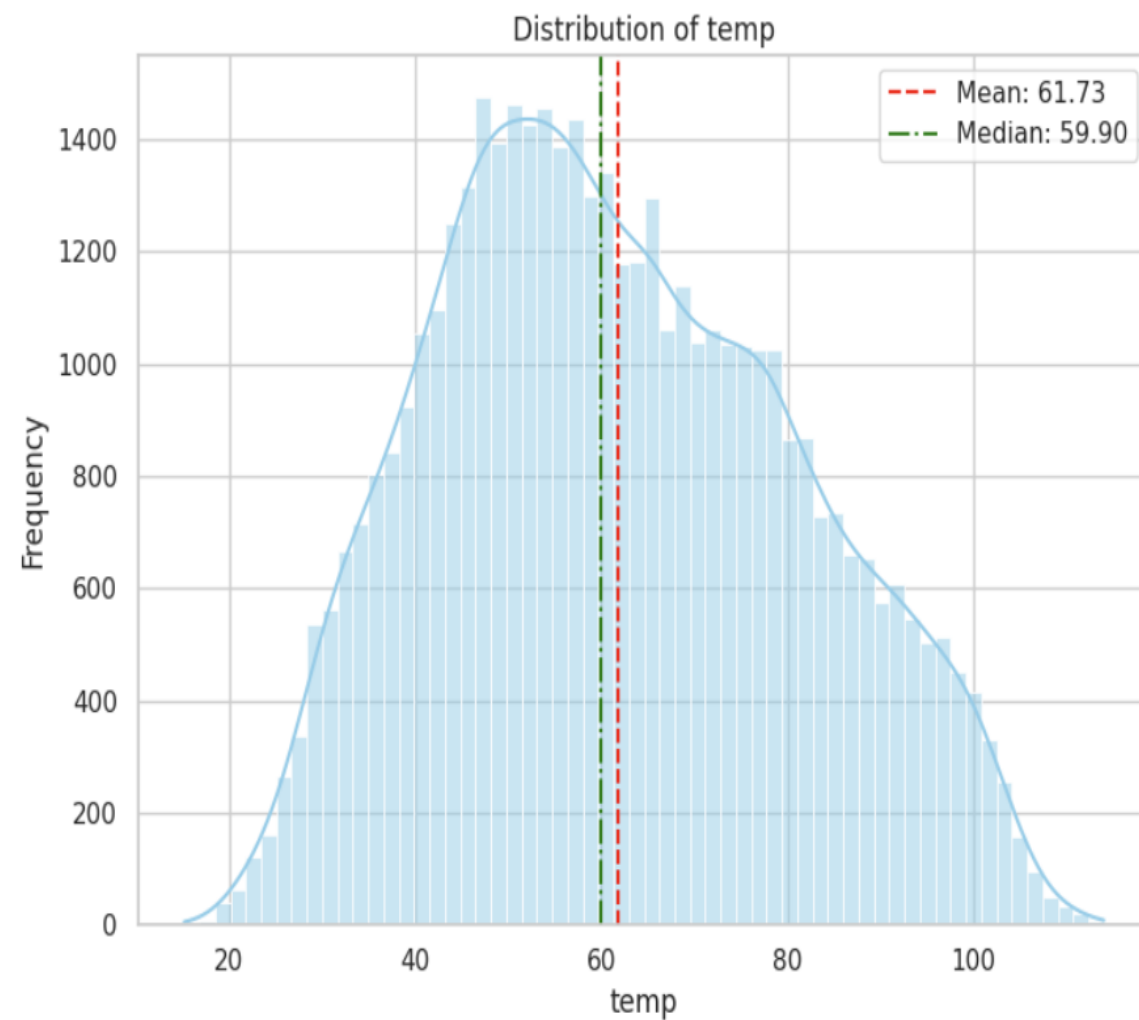
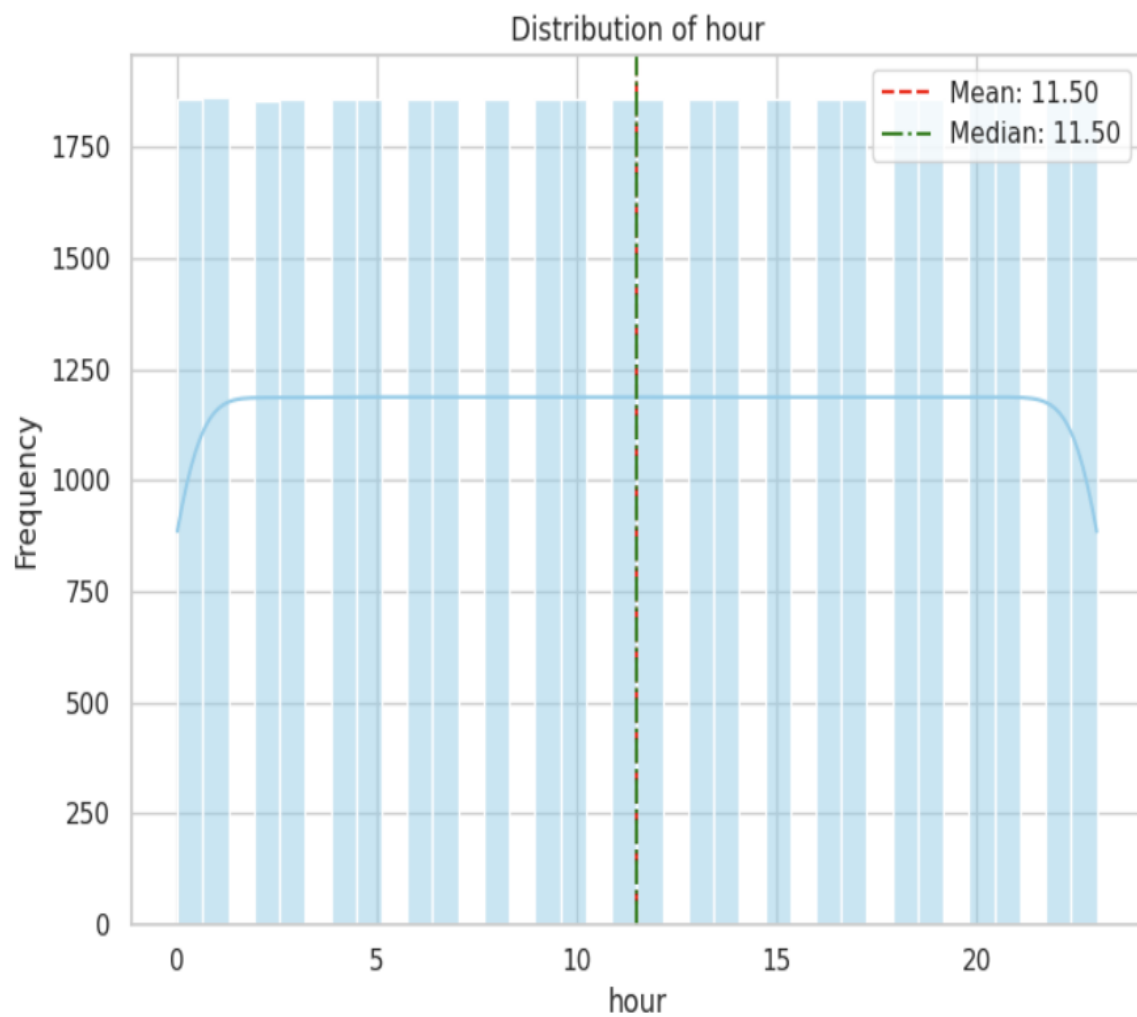
Correlation Matrix of Merged Data



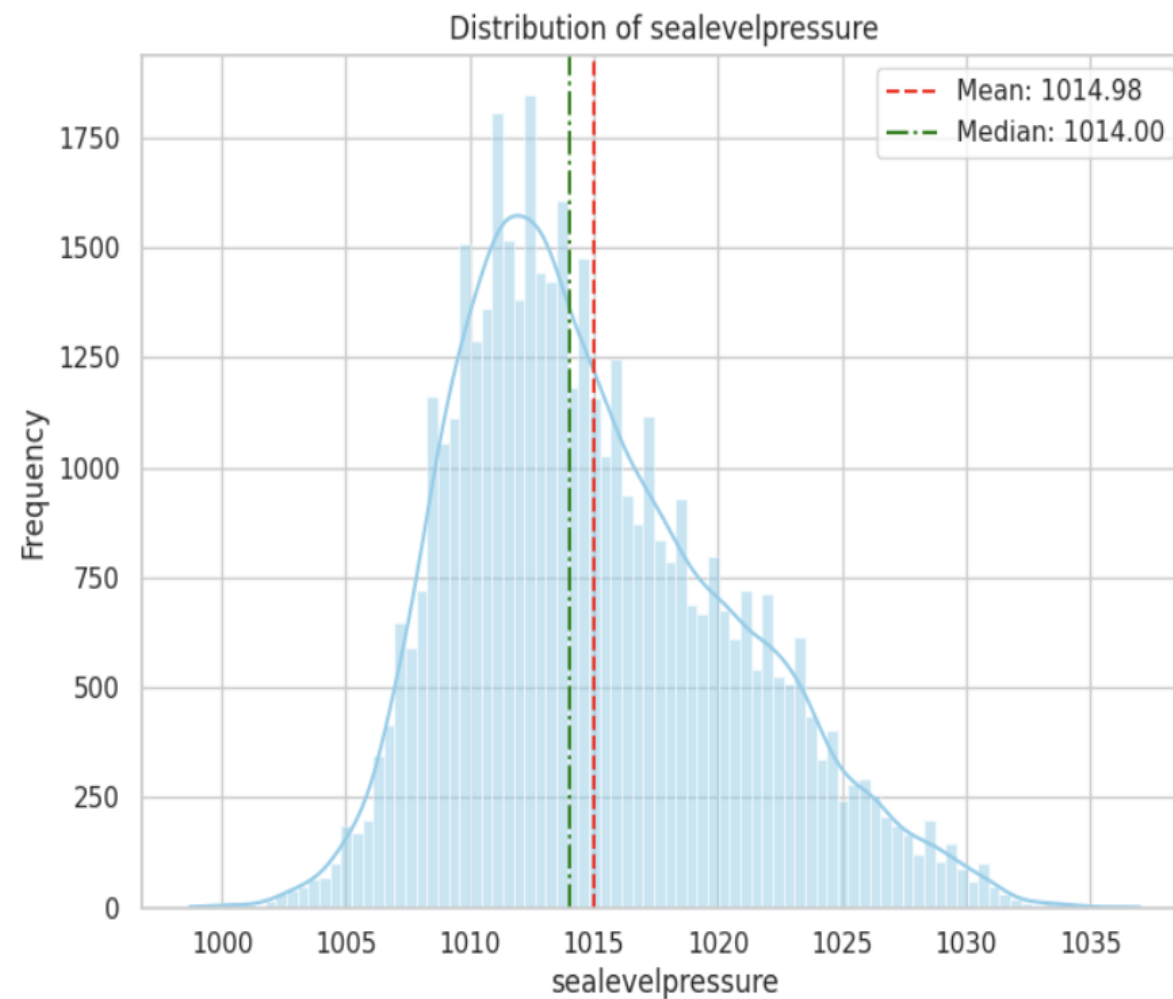
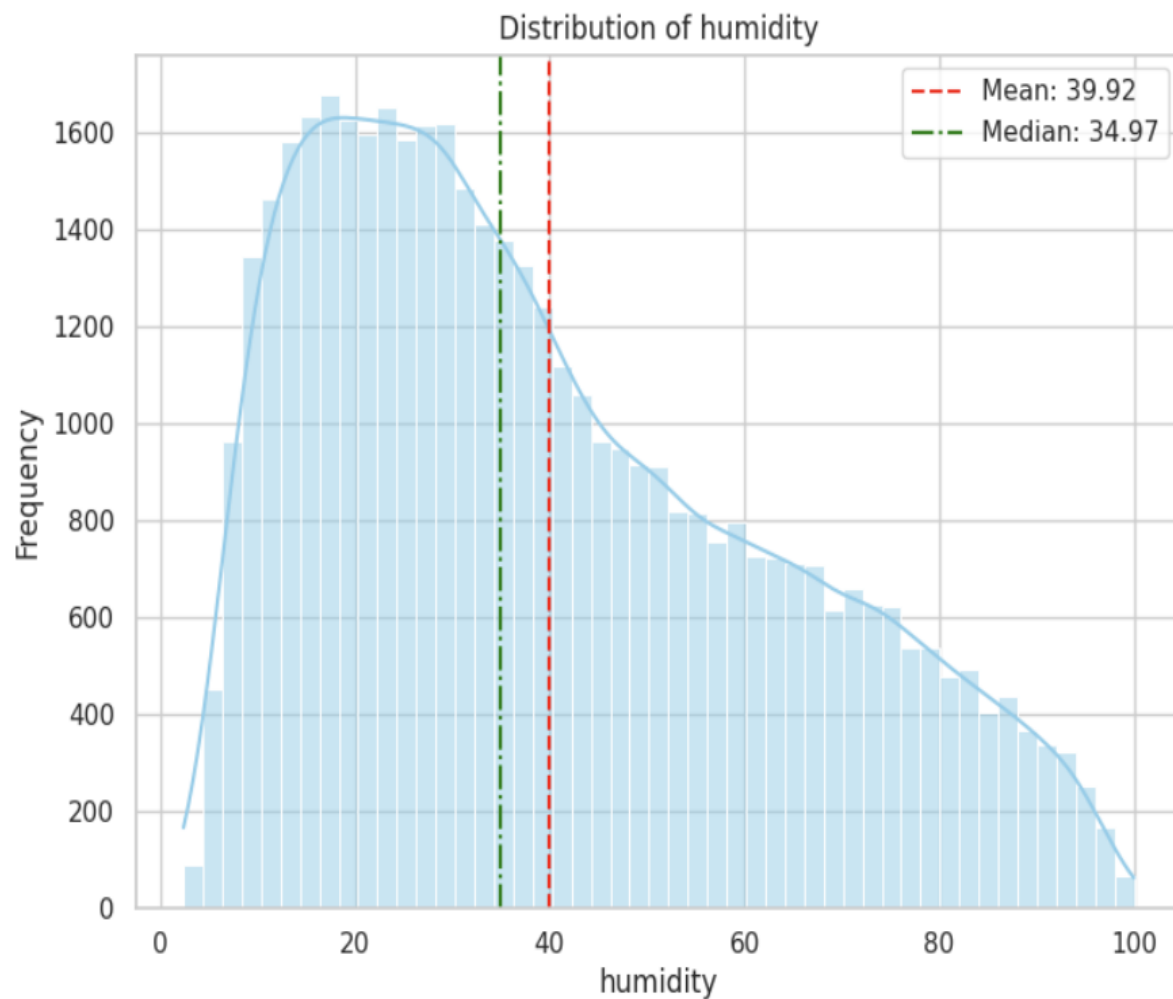
Key Features and Target Variable

- We have identified the feature Load as target Variable.
- To model “load” while addressing multicollinearity, we first identify variables from the list with significant correlations to ‘load’ ($\geq |0.15|$) and then check for high inter-correlations ($\geq |0.7|$) among them.
- Final Variables to Include:
 - hour - Captures the daily load pattern.
 - temp - Kept as required; represents the main temperature effect.
 - humidity - Provides complementary moisture information (kept instead of dew).
 - solar energy - Serves as the representative of solar-related features.
 - sea level pressure

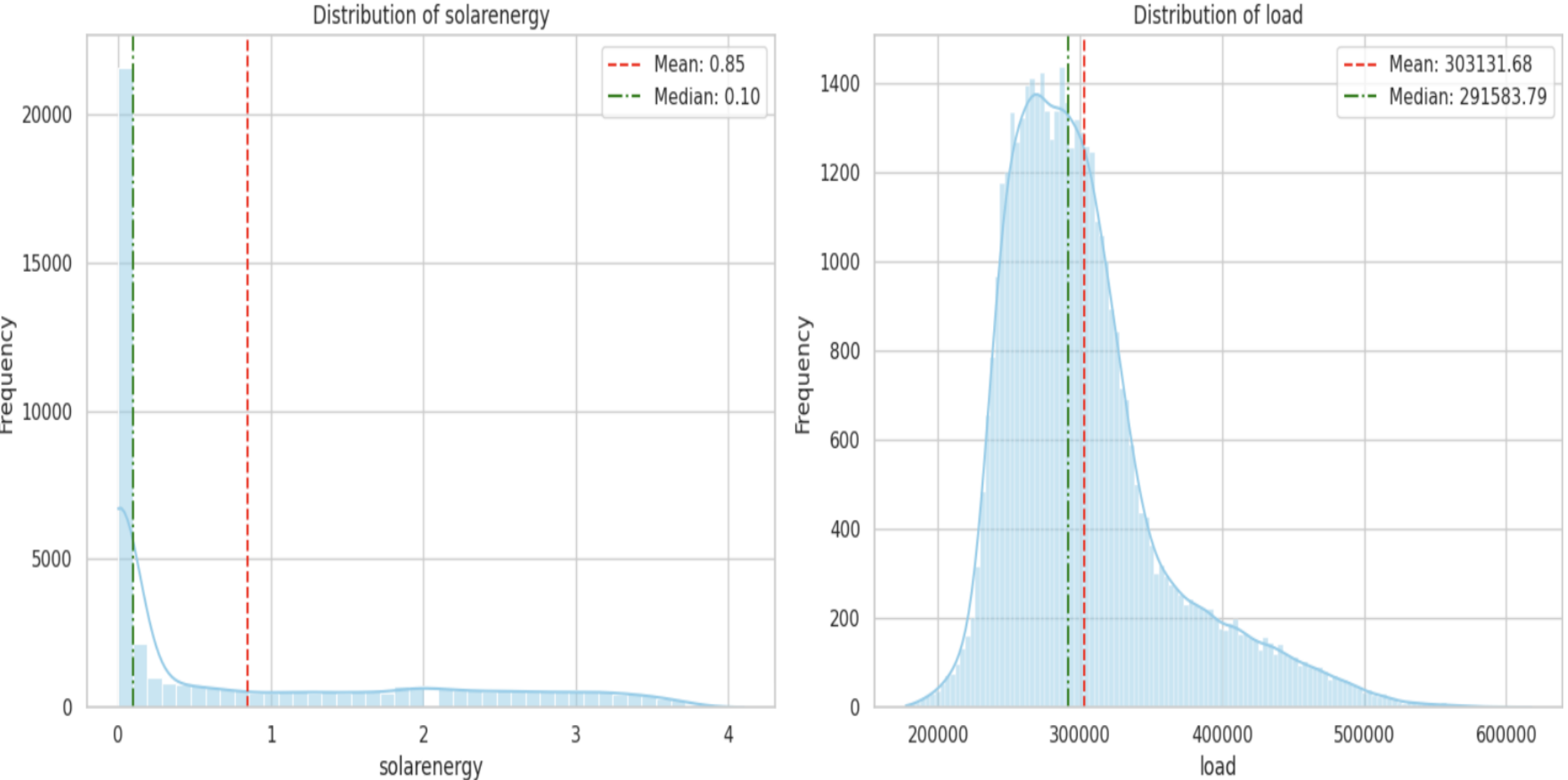
Distribution of Features and Target Variable



Distribution of Features and Target Variable



Distribution of Features and Target Variable



Model Construction

Models Used:

- Linear regression
- Polynomial Regression (degree 2)
- Decision Tree Regression
- Bagging Regressor
- Stacking Regressor
- Extreme Gradient Boosting Regressor
- Facebook Prophet

Data Splitting:

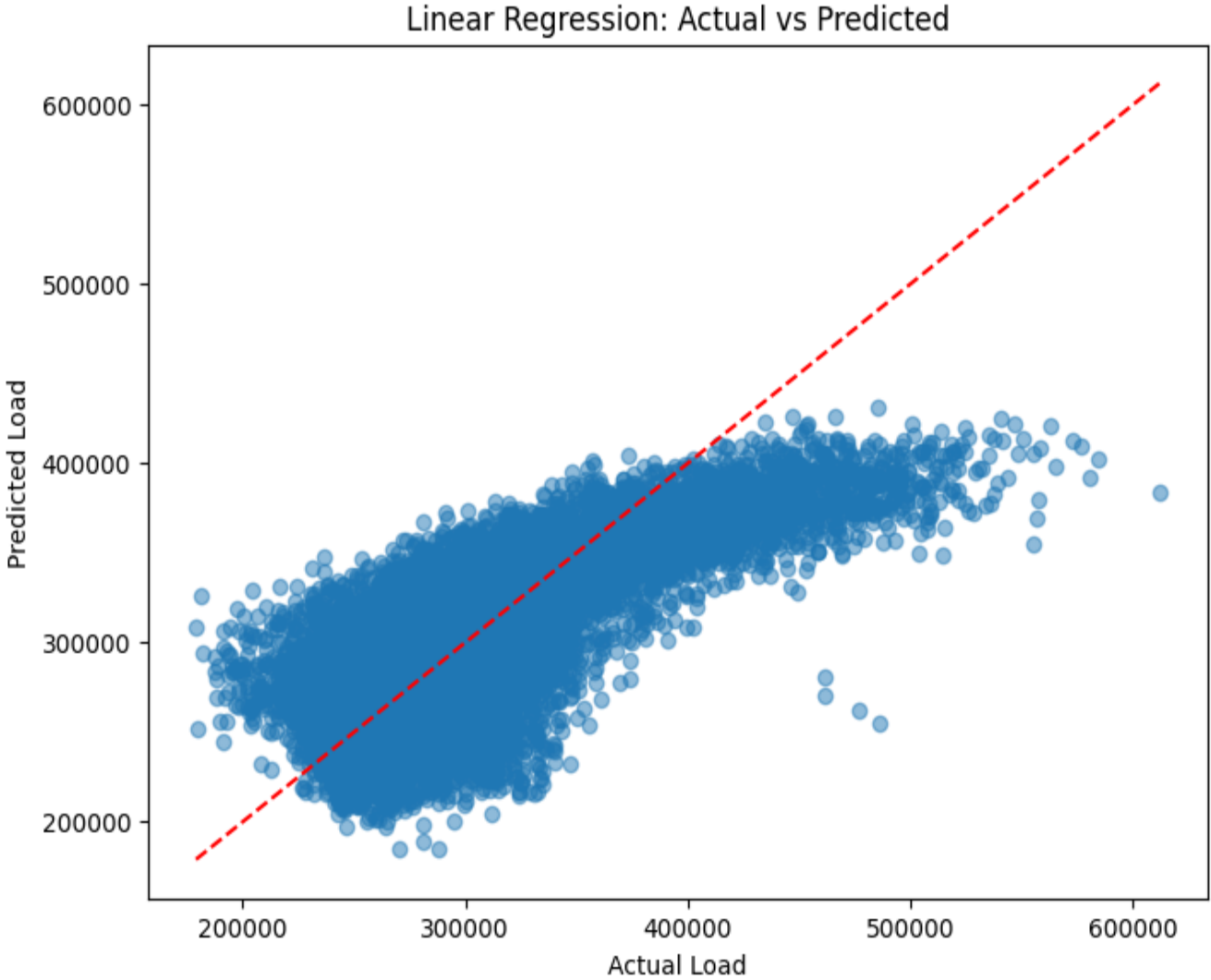
Split the dataset into training and testing (70%-30%)

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=3)
```

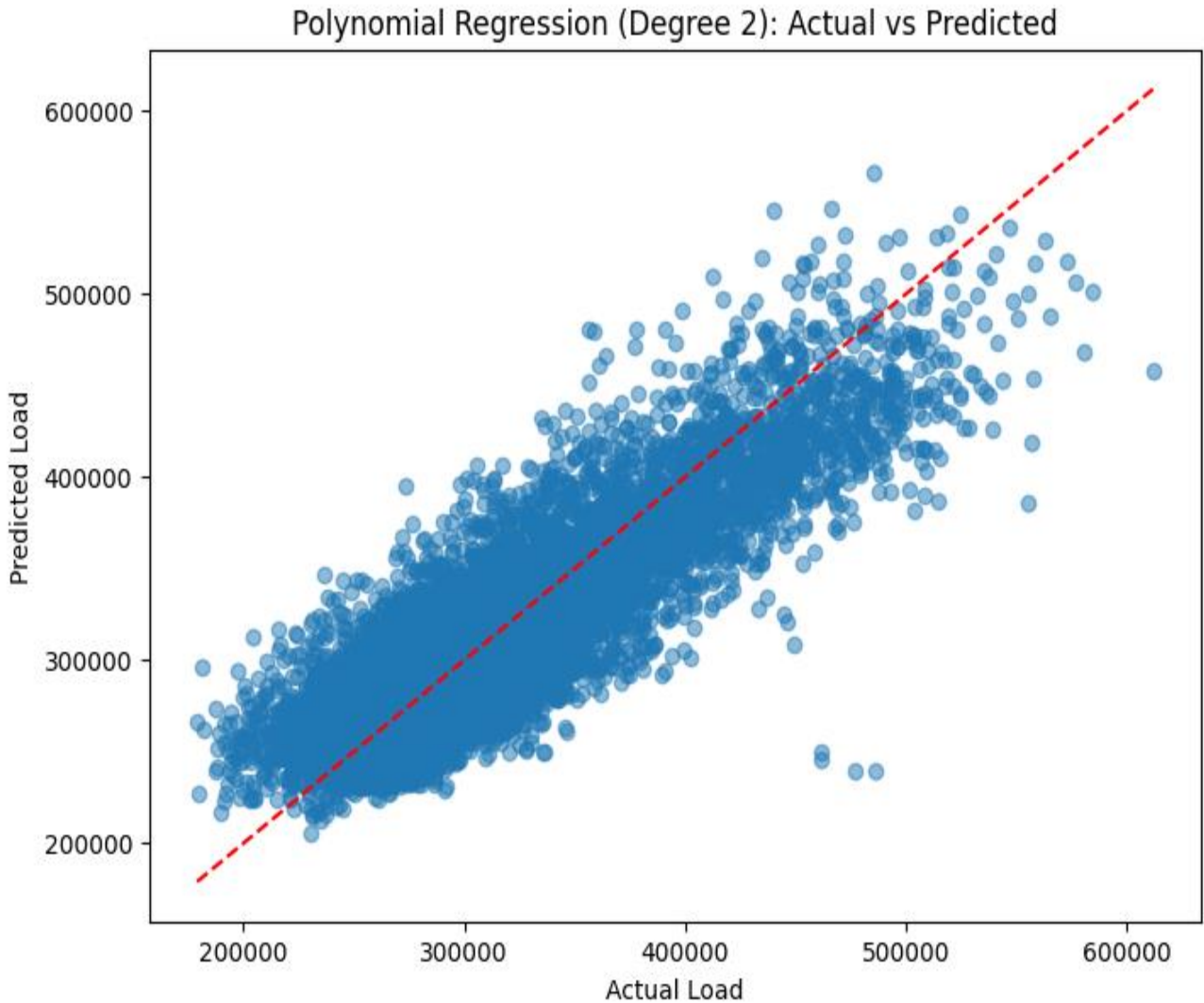
Linear Regression

Model Type	Linear Regression
Train R2	0.5562615249323287
Test R2	0.5464798241814917
Train MAE	29836.940985457735
Test MAE	29970.46640058019



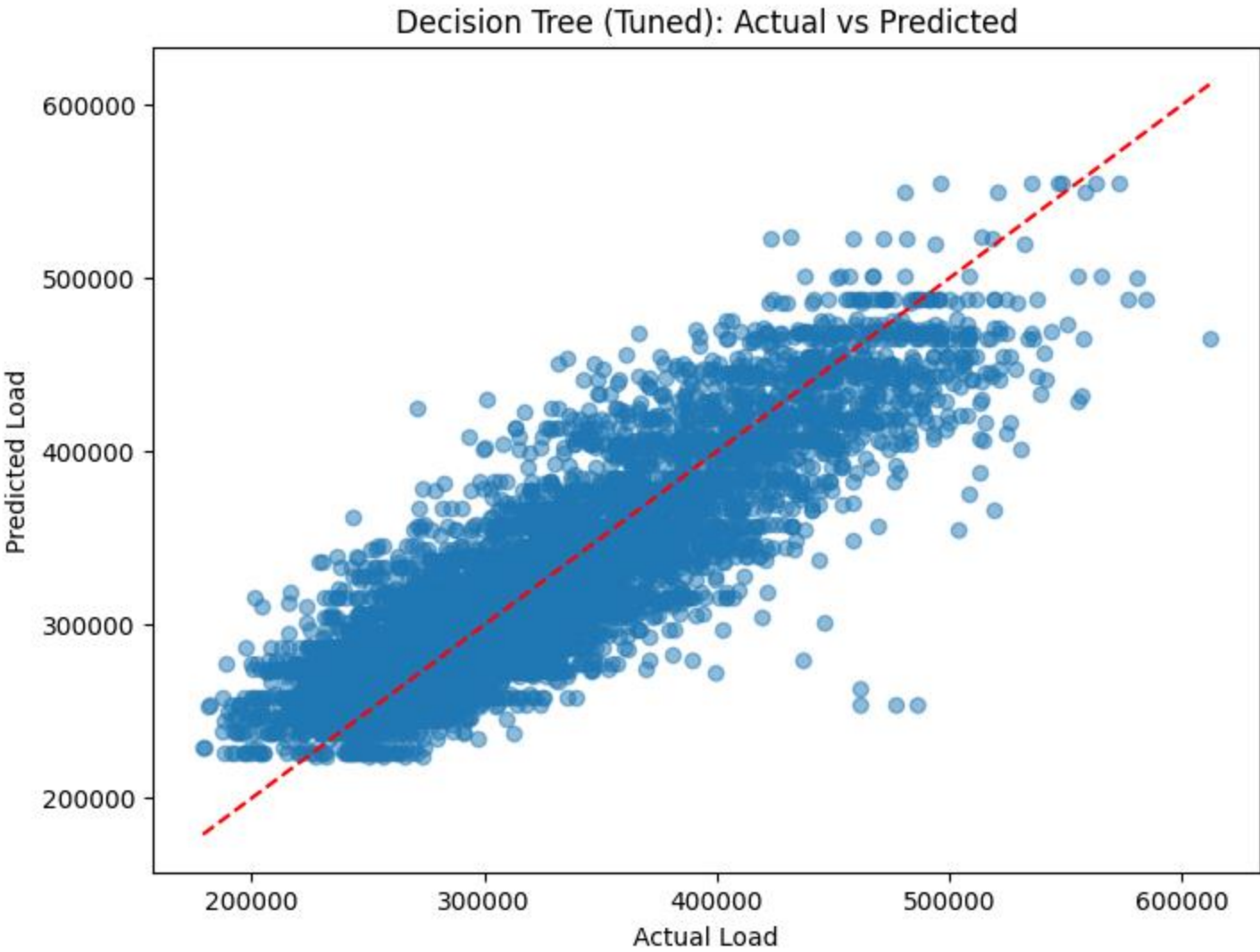
Polynomial Regression (degree 2)

Model Type	Polynomial Regression
Train R2	0.7619550592027355
Test R2	0.755468090917026
Train MAE	21634.189990759915
Test MAE	21804.65843563268



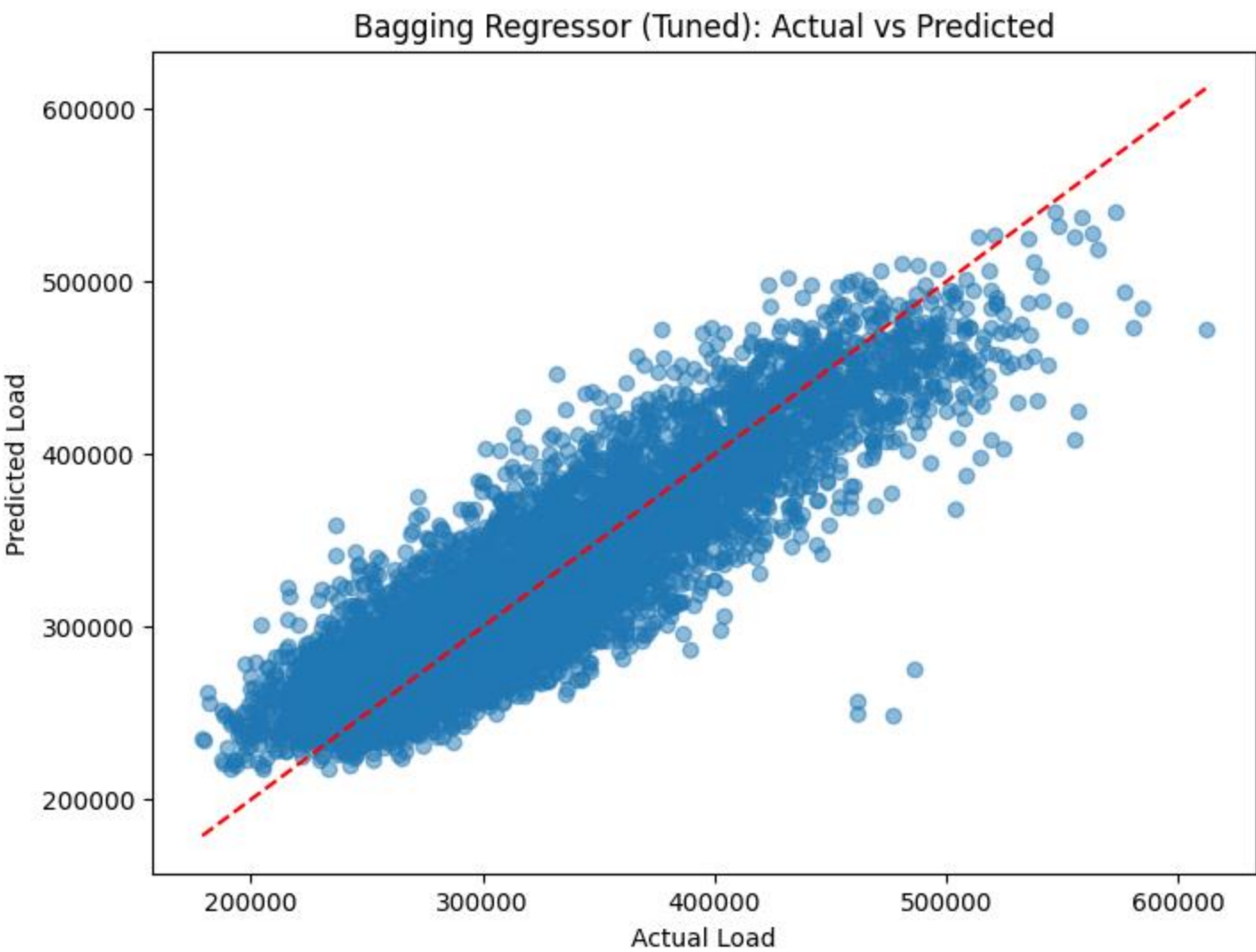
Decision Tree Regression

Model Type	Decision Tree Regression
Train R2	0.8251128345114886
Test R2	0.7863678267727531
Train MAE	17834.141209257938
Test MAE	19369.015492437713



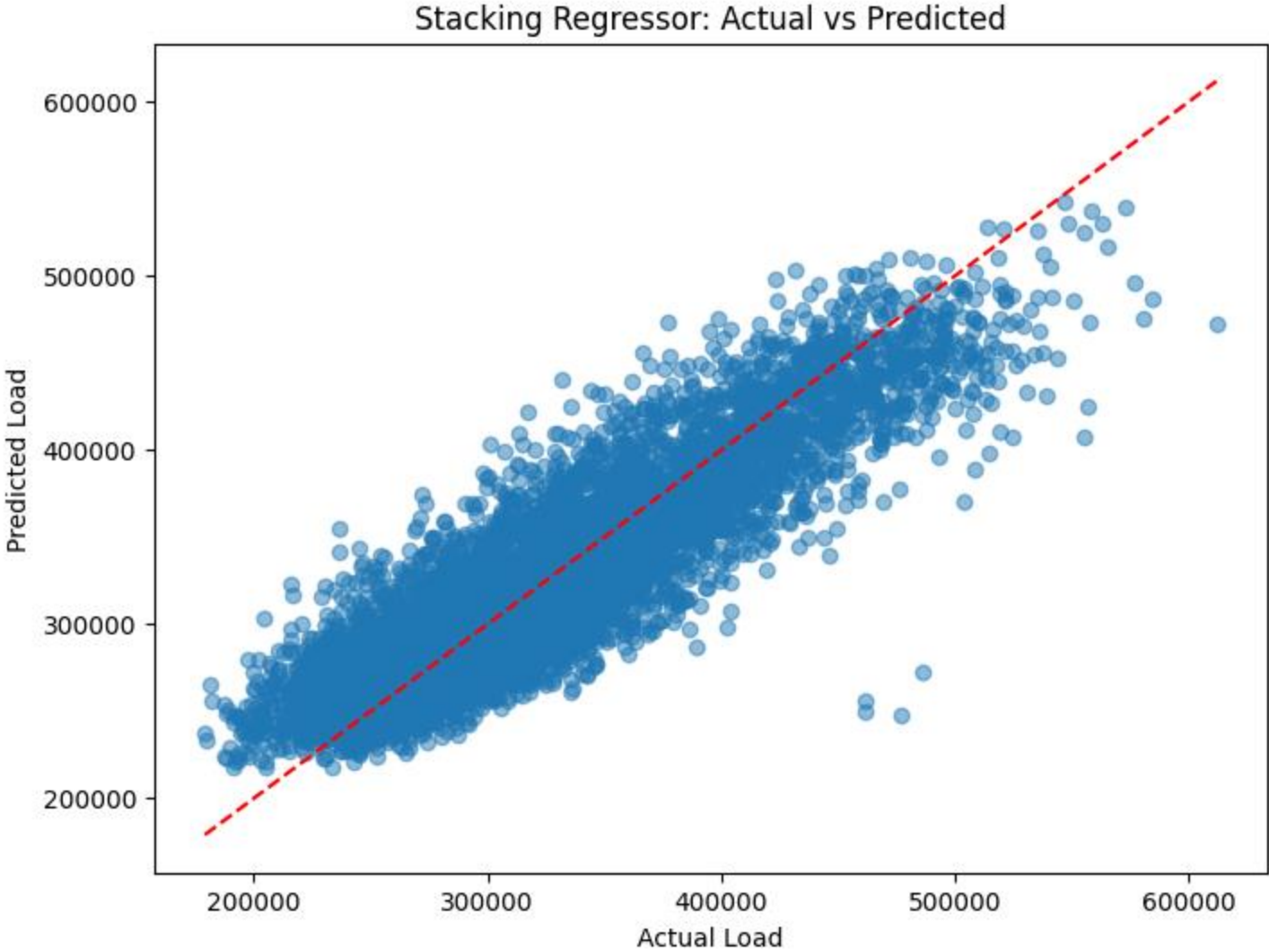
Bagging Regressor

Model Type	Bagging Regressor
Train R2	0.9323077129321571
Test R2	0.8154696003196291
Train MAE	10985.874866196766
Test MAE	18135.388514534297



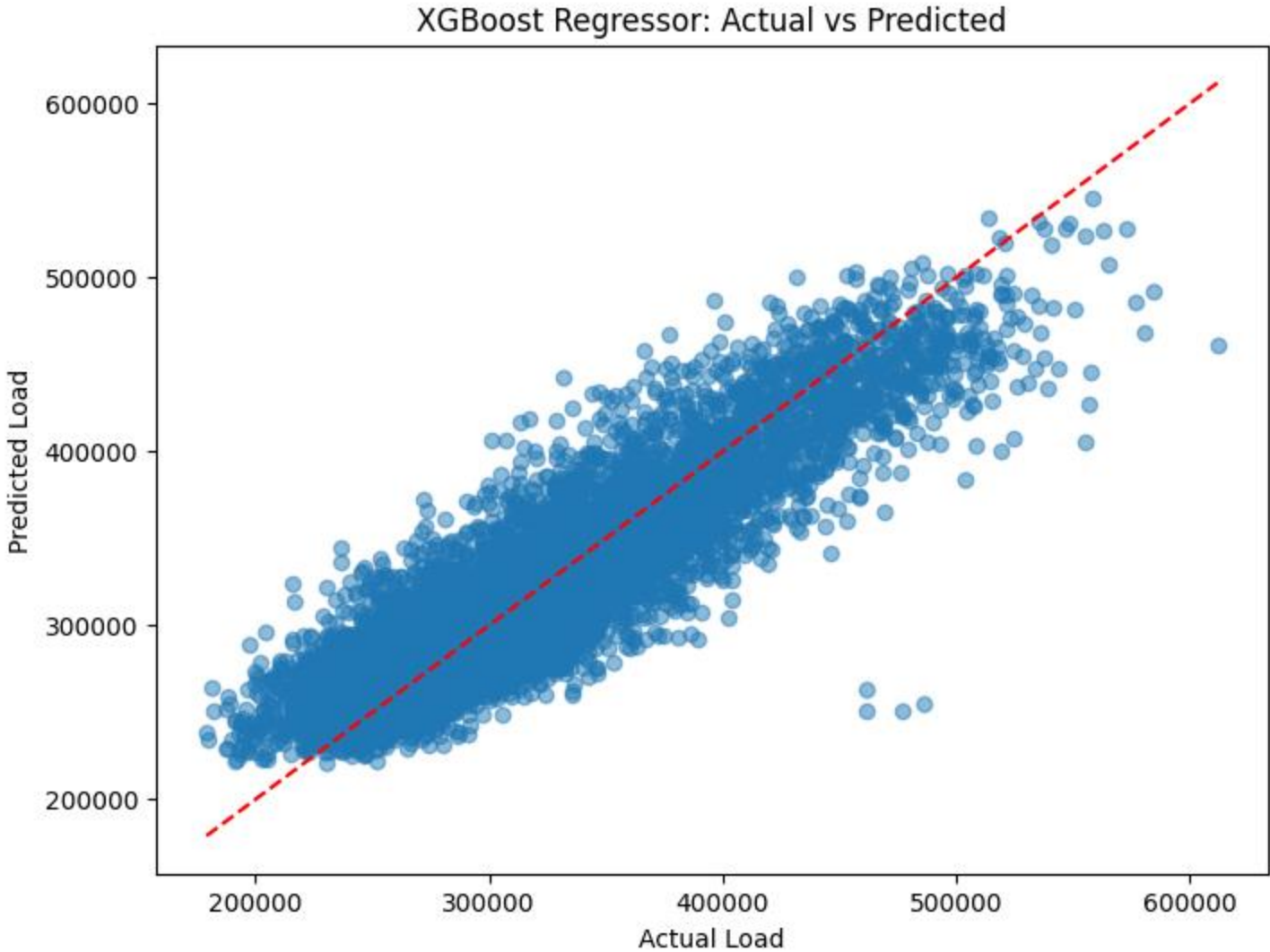
Stacking Regressor

Model Type	Stacking Regressor
Train R2	0.9227549869217017
Test R2	0.8159356489394328
Train MAE	11772.472199138067
Test MAE	18114.714665848096



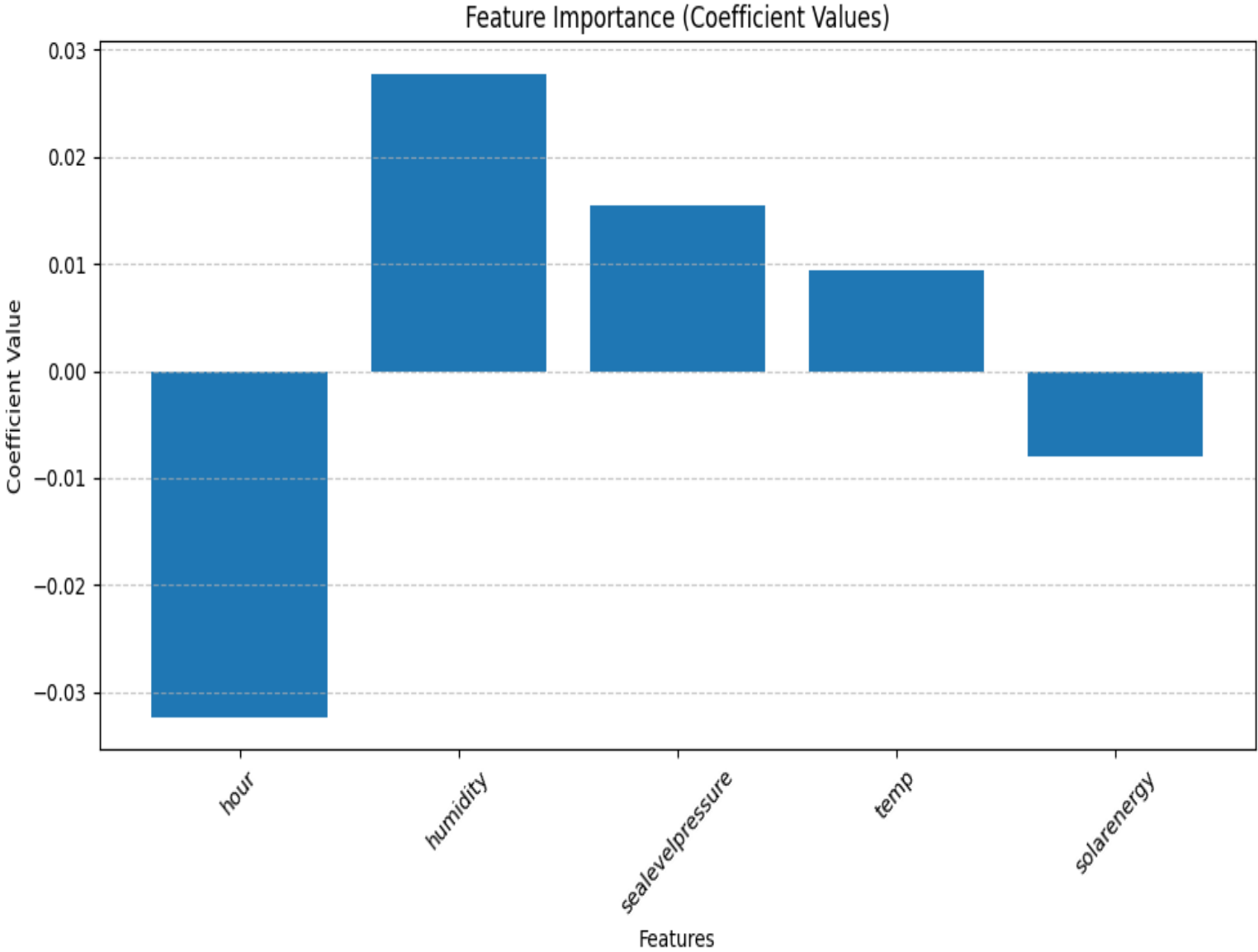
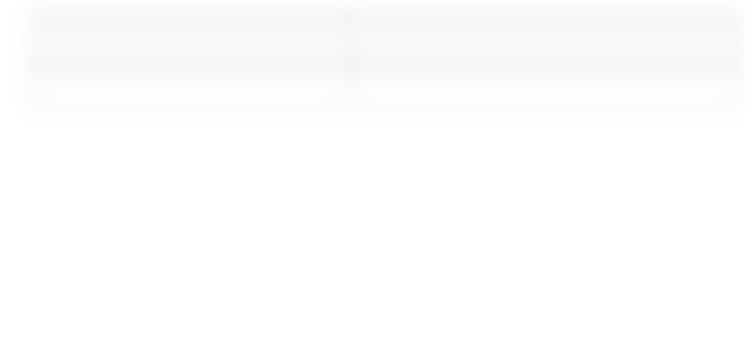
Extreme Gradient Boosting Regressor

Model Type	XG Boosting Regressor
Train R2	0.8647894034141625
Test R2	0.8220134250781448
Train MAE	15697.444074720135
Test MAE	17801.454534340857

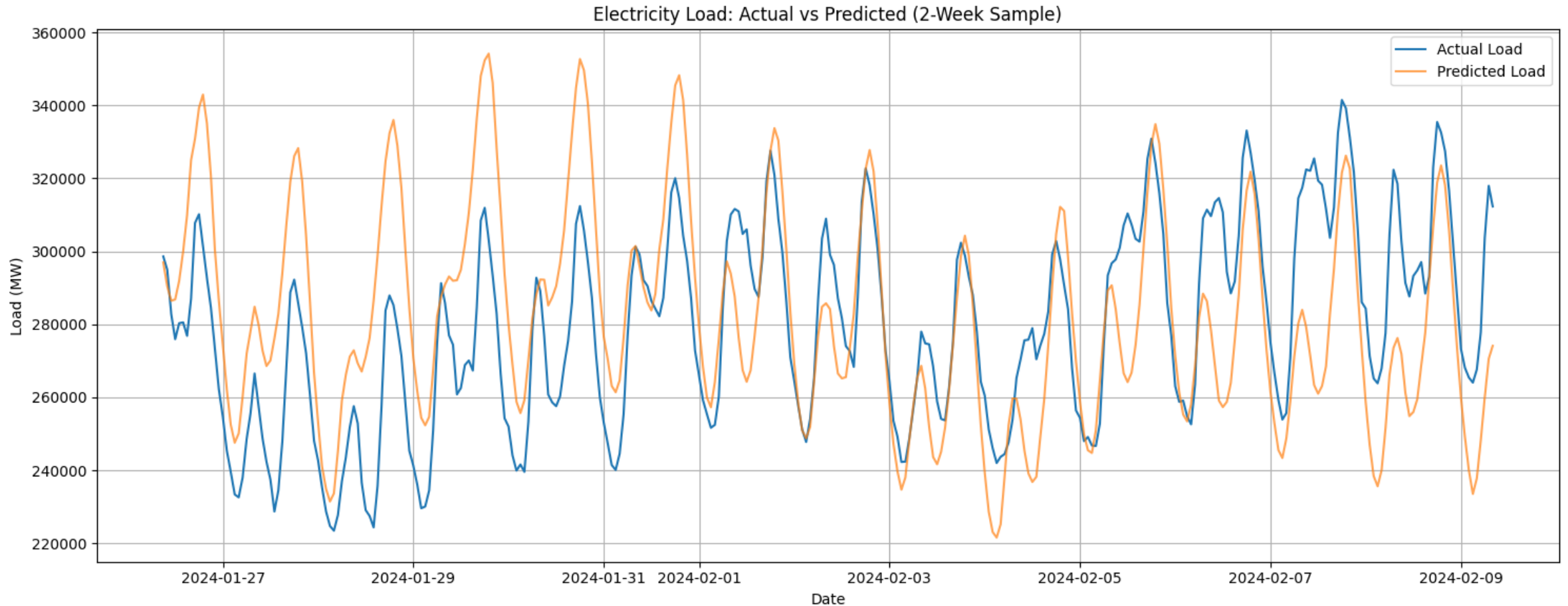


Facebook Prophet

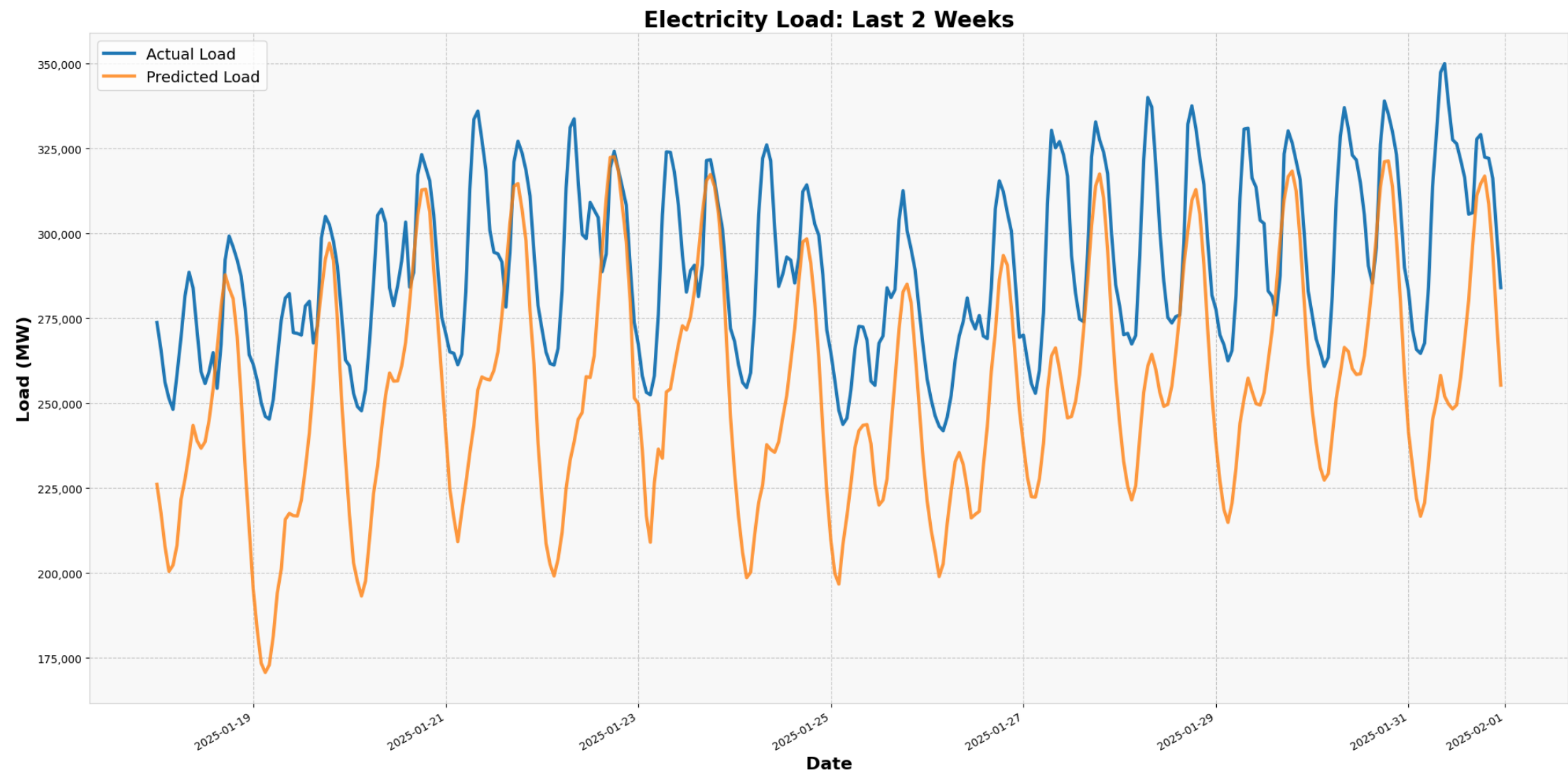
Model Type	Prophet
R2	0.5702
MAE	29784.10 MW
RMSE	38581.28 MW



Facebook Prophet



Facebook Prophet

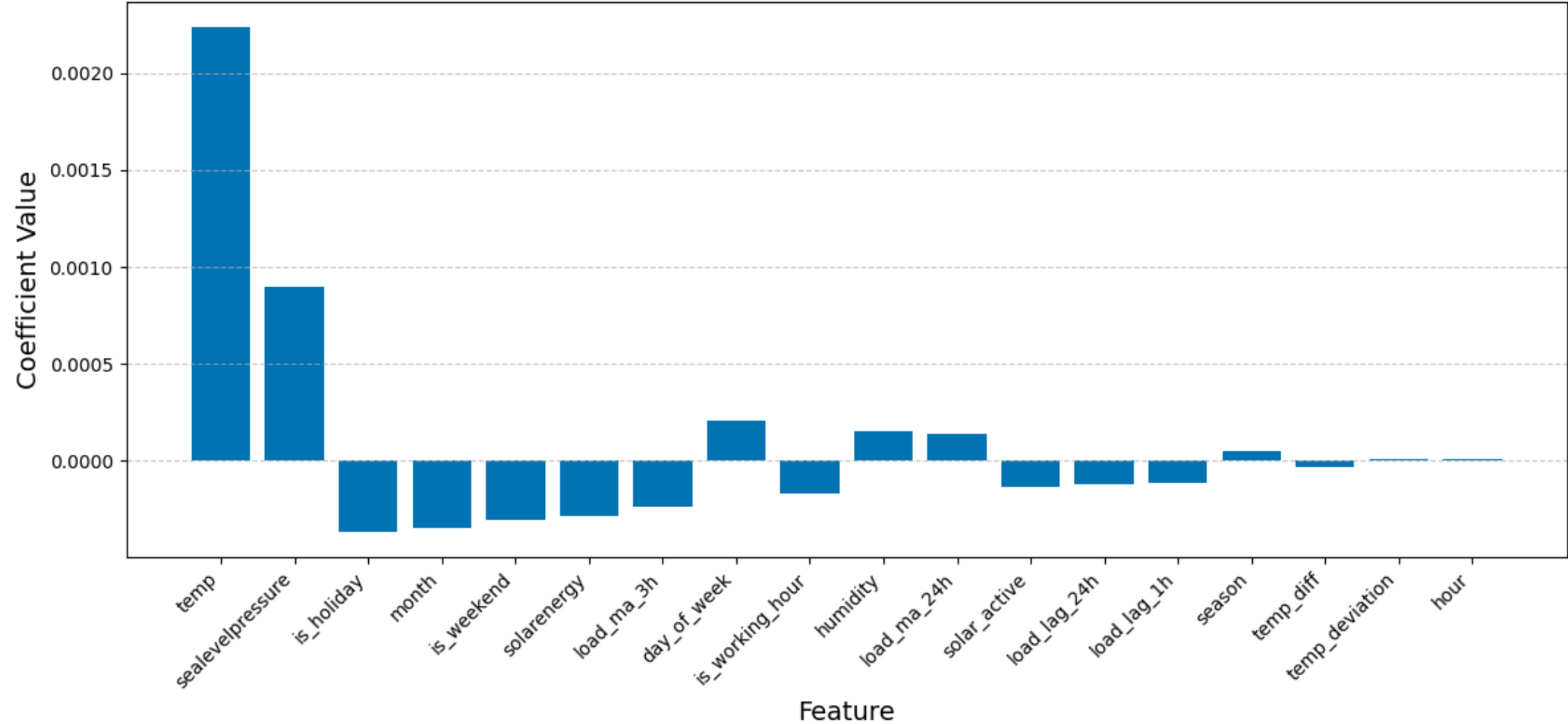


Facebook Prophet Extra Features

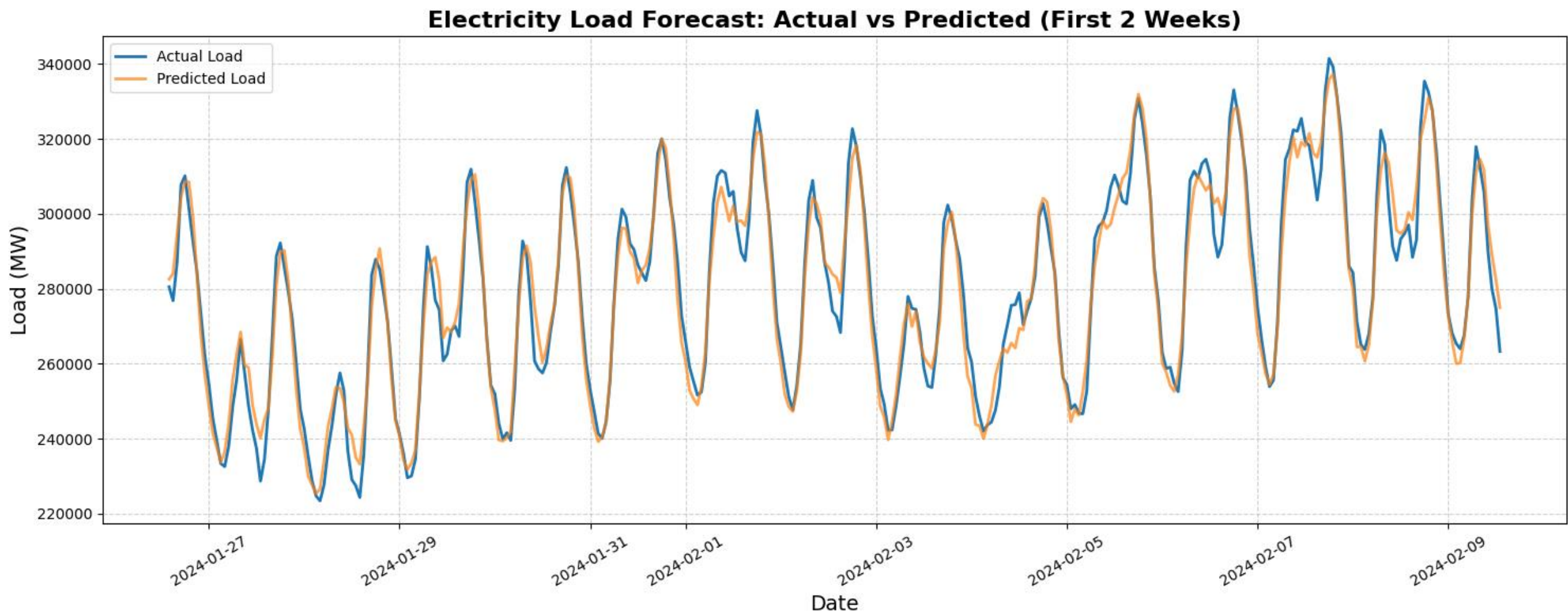
Model Type	Prophet
R2	0.9831
MAE	5693.22 MW
RMSE	7643.54 MW

Facebook Prophet Extra Features

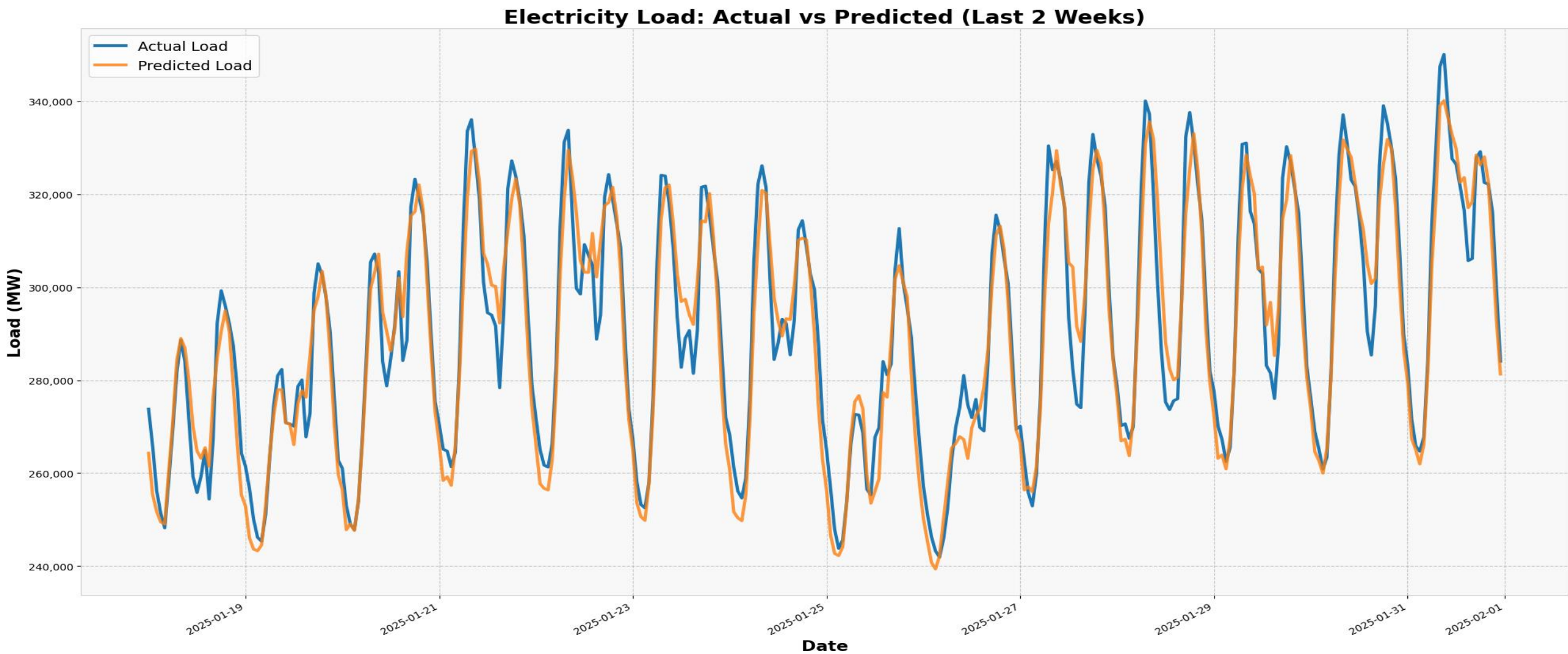
Feature Importance (Prophet Coefficients)



Facebook Prophet Extra Features



Facebook Prophet Extra Features



Model Performance Table

Model	R^2	MAE
Linear Regression	0.546	29,970
Polynomial Regression	0.755	21804
Decision Tree	0.786	19,369
Bagging Regressor	0.815	18,135
Stacking Regressor	0.815	18,114
XGBoost Regressor	0.822	17801
Prophet	0.570	29,784
Prophet Extra Features	0.983	5,693

Conclusion (Model Performance Summary)

Best Performing Model: Prophet Extra Features

- Highest Test R^2 (0.983) – Best variance explanation on unseen data.
- Lowest Test MAE (5,693) – Most accurate predictions.

Future Implementations

- Integration of deep learning models (LSTM, Transformer-based architectures)
- Real-time forecast updates based on new data
- Grid optimization strategies based on forecasts
- Economic impact analysis and cost optimization
- Integration with renewable energy forecasting for grid balancing.

References

1. Efficient Wind Power Prediction Using Machine Learning Methods: A Comparative Study. <https://www.mdpi.com/1996-1073/15/7/2327>
2. Integration of Renewable Energy Sources in future power systems: The role of storage. Authors: Stefan Weitemeyer, David Kleinhans, Thomas Vogt, Carsten Ager. <https://www.sciencedirect.com/science/article/abs/pii/S096014811400593X>
3. Forecasting Renewable Energy Generation with Machine Learning and Deep Learning: Current Advances and Future Prospects. Authors: Natei Ermais , Mesfi. https://www.researchgate.net/publication/370208950_Forecasting_Renewable_Energy_Generation_with_Machine_learning_and_Deep_Learning_Current_Advances_and_Future_Prospects

THANK YOU

ANY QUESTIONS?